

CRUZEIRO DO SUL VIRTUAL

Curso de Ciência da Computação

Disciplina: Projetos Computacionais – da Teoria à Prática

Projeto Integrador Transdisciplinar (PIT) em Ciência da Computação

Catálogo Simples de Filmes

Relatório Técnico do Projeto

Aplicação web de catálogo de filmes (CRUD completo + busca),
desenvolvida em Java
com Servlets, JSP e JDBC, seguindo a metodologia de Aprendizagem
Baseada em Projetos (ABP)

Autor(a):

Antonio Gabinio Filho

E-mail de contato: tomgabinio@gmail.com

Cruzeiro do Sul Virtual – 2026

Sumário

1	Introdução	2
1.1	Objetivos	2
1.2	Justificativa	2
2	Metodologia	3
2.1	Aprendizagem Baseada em Projetos (ABP)	3
2.2	Abordagem iterativa e processo interno de especificação	3
2.3	Tecnologias utilizadas	4
3	Modelagem do Sistema	4
3.1	Casos de uso	4
3.2	Diagrama de classes	5
3.3	Modelo de dados (DER e script SQL)	6
4	Desenvolvimento e Implementação	6
4.1	Arquitetura geral	6
4.2	Funcionalidade em destaque: Cadastro de Filme	7
4.3	Funcionalidade em destaque: Busca de Filme	8
4.4	Extensões opcionais: integração TMDB e reestilização com Bootstrap	9
5	Resultados	10
5.1	Listagem de filmes	10
5.2	Cadastro de filme	12
5.3	Detalhe de filme	12
5.4	Edição de filme	14
5.5	Extensão: Descobrir filmes (TMDB)	14
5.6	Extensão: busca no TMDB dentro do cadastro	16
6	Considerações sobre Segurança	16
6.1	Revisão de segurança por funcionalidade	17
6.2	Achados do pentest <i>blackbox</i> (Nível 3 – trilha de segurança)	17
6.3	Cobertura de testes	18
7	Conclusões	18
7.1	Dificuldades encontradas	18
7.2	Aprendizados	19
7.3	Trabalhos futuros	19
8	Referências	20
8.1	Bibliografia da disciplina	20
8.2	Documentação técnica consultada	21
9	Documentação do Código-Fonte	21
10	Manual do Usuário Simplificado	22

1 Introdução

Neste relatório, descrevo o desenvolvimento do **Catálogo Simples de Filmes**, meu projeto individual da disciplina *Projetos Computacionais: da Teoria à Prática* (Projeto Integrador Transdisciplinar – PIT – em Ciência da Computação, Cruzeiro do Sul Virtual), conduzida pela metodologia de Aprendizagem Baseada em Projetos (ABP). A proposta da disciplina era, de forma genérica, um “Catálogo Simples de Livros ou Filmes”: uma aplicação web capaz de catalogar e consultar itens de mídia, com atributos mínimos de título, autor/diretor, ano de publicação/lançamento, gênero e sinopse.

Optei por um recorte específico dessa proposta genérica: um catálogo focado **exclusivamente em filmes**, com a entidade central **Filme**. Escolhi a temática de cinema, em vez de um catálogo multi-mídia (livros, séries e filmes ao mesmo tempo), como uma decisão consciente de simplificação de escopo, compatível com a recomendação do próprio enunciado de manter o projeto “gerenciável dentro do cronograma semestral”.

1.1 Objetivos

Meu objetivo principal foi aplicar, na prática, os conhecimentos de Programação Orientada a Objetos em Java a um sistema web completo, contemplando:

- Interface web para navegação e gerenciamento do catálogo;
- Cadastro de novos filmes;
- Listagem de todos os filmes catalogados;
- Visualização dos detalhes de um filme específico;
- Edição das informações de um filme já cadastrado;
- Exclusão de um filme do catálogo;
- Busca simples por título ou diretor;
- Persistência dos dados em banco de dados relacional.

Além do escopo mandatório do enunciado, optei por implementar duas extensões opcionais mencionadas como “possíveis extensões” no PDF do enunciado: (a) a integração com a API pública do TMDb (*The Movie Database*), permitindo importar pôsteres, notas públicas e dados de filmes populares diretamente para o catálogo local; e (b) a reestilização completa da interface com o framework Bootstrap 5, incluindo modo escuro alternável e efeitos visuais. Descrevo essas duas extensões em detalhe nas Seções 4 e 5.

1.2 Justificativa

Ainda que o caráter do trabalho seja predominantemente acadêmico, minha justificativa para o projeto é dupla. Do ponto de vista pedagógico, desenvolver um CRUD completo com busca – exatamente o escopo descrito no enunciado – me exercitou de forma concentrada nos pilares do desenvolvimento web em Java (Servlets como controladores, JSP como camada de visão, DAO como camada de acesso a dados via JDBC) e, de forma deliberada, me expôs a três “situações-problema” de segurança e engenharia de software

centrais à disciplina: prevenção de SQL Injection via `PreparedStatement`, tratamento robusto de exceções e validação de dados, e refatoração para reduzir duplicação de código através de camadas DAO/serviço. Do ponto de vista prático, o resultado é uma ferramenta de uso pessoal que efetivamente uso: um catálogo dos filmes que já assisti ou pretendo assistir, enriquecido com dados públicos do TMDb.

2 Metodologia

2.1 Aprendizagem Baseada em Projetos (ABP)

Segui integralmente, no desenvolvimento deste projeto, a metodologia de Aprendizagem Baseada em Projetos (ABP), conforme proposta pela disciplina. A ABP é uma estratégia pedagógica que coloca o(a) estudante no centro do processo de aprendizagem, promovendo a aquisição de conhecimento por meio do engajamento ativo em projetos do mundo real e pessoalmente significativos (LARMER; MERGENDOLLER; BOSS, 2015). Diferentemente de modelos tradicionais de ensino expositivo, a ABP exige investigação sustentada – pesquisa, análise crítica e síntese de informações ao longo de um período extenso (THOMAS, 2000) – e autenticidade, isto é, problemas contextualizados no mundo real, o que aumenta o engajamento e a percepção de relevância do aprendizado (BLUMENFELD et al., 1991). No contexto da Ciência da Computação, essas características se traduziram diretamente no desenvolvimento do sistema funcional que apresento neste relatório, do levantamento de requisitos até a entrega documentada, exercitando tanto habilidades técnicas quanto competências do século XXI – pensamento crítico, criatividade, colaboração e comunicação (TRILLING; FADEL, 2009).

2.2 Abordagem iterativa e processo interno de especificação

Segui uma abordagem **iterativa**, dividida em pequenos marcos (ciclos ou *sprints*) alinhados às funcionalidades do CRUD: módulo de cadastro, módulo de listagem/busca, módulo de detalhe, módulo de edição/exclusão e, por fim, as extensões opcionais (integração TMDb e reestilização de interface). Dentro de cada ciclo, segui um processo de **Spec-Driven Development** (SDD): toda funcionalidade nasceu como uma especificação formal (arquivo em `specs/`, a partir de um modelo padronizado) *antes* de qualquer linha de código, contendo requisitos funcionais e não funcionais, modelo de dados, modelagem UML, critérios de aceite e considerações de segurança. Só avancei cada especificação de fase – Rascunho, Em Revisão, Aprovada, Implementada, Concluída – quando os critérios daquela fase estavam integralmente satisfeitos, incluindo uma revisão de segurança e testes automatizados antes de considerá-la concluída. Esse processo interno de especificação, documentado em `docs/sdd/`, deu rastreabilidade direta entre os requisitos do enunciado da disciplina, o código efetivamente produzido e este Relatório Técnico.

As quatro fases exigidas pelo enunciado da disciplina – concepção e planejamento; desenvolvimento iterativo; testes e refinamento; documentação e apresentação – mapeiam diretamente aos quatro níveis desse processo interno, cada um responsável por um conjunto de entregáveis específico, do levantamento de requisitos e modelagem inicial até este próprio Relatório Técnico.

2.3 Tecnologias utilizadas

Usei a stack tecnológica exigida pelo enunciado da disciplina, sem *frameworks* adicionais além do estritamente necessário para a apresentação (Bootstrap, via CDN, discutido na Seção 4):

- **Linguagem/paradigma:** Java, com Programação Orientada a Objetos – classes de modelo, interfaces de acesso a dados, encapsulamento, e separação clara de responsabilidades entre camadas;
- **Servlets:** controladores (*controllers*) que recebem requisições HTTP, invocam a lógica de negócio/persistência e encaminham a resposta para uma página JSP;
- **JavaServer Pages (JSP):** camada de apresentação (*view*), com Expression Language (EL) e JavaServer Pages Standard Tag Library (JSTL) para exibição segura de dados;
- **Java Database Connectivity (JDBC):** conexão da aplicação Java ao banco de dados relacional, com todo acesso via `PreparedStatement` (nunca concatenação de SQL);
- **Sistema Gerenciador de Banco de Dados (SGBD):** MySQL/MariaDB (driver `mysql-connector-j`), com a tabela `filme` como fonte única de verdade do catálogo (ver Seção 3);
- **JUnit 5:** testes automatizados da camada de acesso a dados (`FilmeDAO`);
- **Apache Maven:** gerenciamento de dependências e build do projeto;
- **Apache Tomcat:** servidor de aplicação (*container* de Servlets) usado para *deploy* e testes locais.

3 Modelagem do Sistema

3.1 Casos de uso

O sistema gira em torno de um único ator, o “Usuário do Catálogo” (sem distinção de papéis, já que a aplicação não implementa autenticação – decisão de escopo discutida na Seção 6). Os casos de uso mandatórios, extraídos das especificações de cada funcionalidade, são:

- **Cadastrar Filme:** pré-condição nenhuma; pós-condição, o filme é persistido na tabela `filme` e o usuário vê uma mensagem de sucesso;
- **Listar Filmes:** pré-condição nenhuma; pós-condição, o usuário vê todos os filmes cadastrados (ou uma mensagem de catálogo vazio);
- **Ver Detalhes do Filme:** pré-condição, um `id` de filme existente; pós-condição, o usuário vê todos os campos daquele filme (ou “Filme não encontrado.”);
- **Editar Filme:** especializa/depende de “Ver Detalhes”; pré-condição, um `id` existente; pós-condição, o filme é atualizado e o usuário retorna ao detalhe com mensagem de sucesso;

- **Excluir Filme:** especializa/depende de “Ver Detalhes”; pré-condição, um id existente e confirmação explícita do usuário; pós-condição, o filme é removido e o usuário retorna à listagem;
- **Buscar Filme por Título/Diretor:** especializa “Listar Filmes” (mesma tela, parâmetro de busca opcional); pré-condição nenhuma; pós-condição, o usuário vê apenas os filmes cujo título ou diretor correspondem ao termo buscado.

Como extensão opcional, implementei dois casos de uso adicionais: **Descobrir Filmes Populares (TMDB)**, que exibe um carrossel de filmes populares consultados em tempo real na API do TMDB, com um botão **Importar Filme do TMDB** que insere o filme escolhido diretamente no catálogo local; e **Buscar Filme no TMDB** dentro do próprio formulário de cadastro, que pré-preenche os campos a partir de um resultado escolhido pelo usuário. Em ambos os casos, decidi que uma eventual indisponibilidade da API do TMDB nunca deveria comprometer o cadastro manual nem o restante do catálogo – decisão de design registrada explicitamente na especificação da integração.

3.2 Diagrama de classes

O diagrama de classes consolidado do projeto (fonte completa em PlantUML disponível em docs/modelagem/diagrama-classes-geral.puml, acumulado incrementalmente a cada funcionalidade especificada) organiza o sistema em cinco grupos principais:

- **Modelo** (com.catalogo.model): a classe `Filme`, um POJO puro com os atributos `id`, `titulo`, `diretor`, `anoLancamento`, `genero`, `sinopse`, `capaUrl` e `notaTmdb`, e os respectivos *getters/setters*;
- **Acesso a dados** (com.catalogo.dao): a interface `FilmeDAO`, definindo o contrato `inserir`, `listarTodos`, `buscarPorId`, `atualizar`, `excluir`, `buscarPorTituloOuDiretor` e `existeComTituloEAno`; sua implementação `JDBC FilmeDAOImpl`; e a classe utilitária `FabricaDeConexoes`, responsável por abrir conexões configuradas via arquivo de propriedades (nunca credenciais no código-fonte – ver Seção 6);
- **Serviço** (com.catalogo.service): a classe `FilmeValidator`, compartilhada entre o cadastro e a edição, centralizando a validação de dados de entrada antes de qualquer persistência;
- **Controllers** (com.catalogo.servlet): sete Servlets, um por operação do CRUD/busca/integração TMDB (`CadastrarFilmeServlet`, `ListarFilmesServlet`, `DetalharFilmeServlet`, `EditarFilmeServlet`, `ExcluirFilmeServlet`, `DescobrirFilmesServlet`, `ImportarFilmeTmdbServlet`), além de dois filtros transversais (`CodificacaoUtf8Filter`, para forçar UTF-8 em toda requisição/resposta, e `CabecalhosSegurancaFilter`, discutido na Seção 6);
- **Integração TMDB** (com.catalogo.tmdb, extensão opcional): as classes `TmdbClient` (cliente HTTP/JSON usando `java.net.http.HttpClient` do próprio JDK), `TmdbFilme` (DTO dos dados retornados pela API), `FabricaTmdb` (leitura da chave de API) e `TmdbException`.

As relações principais são: `FilmeDAOImpl` implementa `FilmeDAO`; cada Servlet do CRUD depende de `FilmeDAO` (nunca acessa o banco diretamente); `CadastrarFilmeServlet` e `EditarFilmeServlet` dependem também de `FilmeValidator`; e, na extensão TMDb, `DescobrirFilmesServlet` e `ImportarFilmeTmdbServlet` dependem de `TmdbClient`, com `ImportarFilmeTmdbServlet` delegando a inserção efetiva a `FilmeDAO` – ou seja, a importação do TMDb reaproveita o mesmo caminho de persistência do cadastro manual, sem duplicar lógica de acesso a dados.

3.3 Modelo de dados (DER e script SQL)

O catálogo é persistido em uma única tabela, `filme`, cuja DDL canônica (fonte única de verdade, referenciada por todas as especificações do projeto) é reproduzida a seguir:

```
1 CREATE TABLE filme (  
2   id INT PRIMARY KEY AUTO_INCREMENT,  
3   titulo VARCHAR(255) NOT NULL,  
4   diretor VARCHAR(255),  
5   ano_lancamento INT,  
6   genero VARCHAR(100),  
7   sinopse TEXT,  
8   capa_url VARCHAR(500),  
9   nota_tmdb DECIMAL(3,1)  
10 );
```

Código-Fonte 1: DDL canônica da tabela `filme` (`docs/modelagem/der.sql`)

Apenas `titulo` é obrigatório (`NOT NULL`); todos os demais campos são opcionais, refletindo minha decisão de projeto de que um cadastro mínimo (só o título) já é válido. As colunas `capa_url` e `nota_tmdb` foram adicionadas em um segundo momento, via `ALTER TABLE`, para suportar a extensão de integração com o TMDb, sem exigir nenhuma tabela adicional – os dados do TMDb são consultados em tempo real e apenas os filmes efetivamente importados são persistidos localmente.

4 Desenvolvimento e Implementação

4.1 Arquitetura geral

A aplicação segue a arquitetura em camadas exigida pelo enunciado: **JSP** (visão) → **Servlet** (controlador) → **Service** (regra de negócio, quando aplicável) → **DAO** (acesso a dados via JDBC) → **banco de dados**. Cada operação do CRUD tem seu próprio Servlet, mapeado a uma rota fixa (`/cadastrarFilme`, `/listarFilmes`, `/detalharFilme`, `/editarFilme`, `/excluirFilme`), seguindo o fluxo padrão descrito no material da disciplina: a JSP exibe um formulário HTML; o usuário preenche e submete; o Servlet recebe os dados via `request.getParameter()`, valida, monta um objeto `Filme` e invoca o método correspondente em `FilmeDAO`, que usa `PreparedStatement` para executar o comando SQL; por fim, o Servlet redireciona para uma página de sucesso (listagem ou detalhe) ou reexibe o formulário com mensagens de erro.

Tomei duas decisões de arquitetura para reduzir a duplicação de código, no espírito da terceira situação-problema do enunciado (refatoração e orientação a objetos): (a) extraí a validação de entrada para a classe de serviço `FilmeValidator`, compartilhada entre o cadastro e a edição, em vez de duplicá-la em cada Servlet; e (b) não criei uma tela

separada para a busca – ela estende `ListarFilmesServlet/listarFilmes.jsp` com um parâmetro opcional `termo`, já que conceitualmente é a mesma listagem, apenas filtrada.

4.2 Funcionalidade em destaque: Cadastro de Filme

O cadastro é a operação fundacional do CRUD. O trecho a seguir mostra o método `doPost` de `CadastrarFilmeServlet`, incluindo a verificação de duplicidade (mesmo título e ano de lançamento já cadastrados) que adicionei em uma iteração posterior do projeto:

```
1 protected void doPost(HttpServletRequest request, HttpServletResponse response)
2     throws ServletException, IOException {
3
4     String titulo = FilmeValidator.tratarEntrada(request.getParameter("titulo"));
5     String diretor = FilmeValidator.tratarEntrada(request.getParameter("diretor"));
6     String anoLancamentoStr = FilmeValidator.tratarEntrada(request.getParameter("
7         anoLancamento"));
8     String genero = FilmeValidator.tratarEntrada(request.getParameter("genero"));
9     String sinopse = FilmeValidator.tratarEntrada(request.getParameter("sinopse"));
10    String capaUrl = FilmeValidator.tratarEntrada(request.getParameter("capaUrl"));
11
12    List<String> erros = FilmeValidator.validar(titulo, diretor, anoLancamentoStr,
13        genero, capaUrl);
14
15    // Recebe os valores preenchidos em caso de erro, para o usuário não perder o que
16    // digitou.
17    request.setAttribute("titulo", titulo);
18    request.setAttribute("diretor", diretor);
19    request.setAttribute("anoLancamento", anoLancamentoStr);
20    request.setAttribute("genero", genero);
21    request.setAttribute("sinopse", sinopse);
22    request.setAttribute("capaUrl", capaUrl);
23
24    if (!erros.isEmpty()) {
25        request.setAttribute("erros", erros);
26        request.getRequestDispatcher("/WEB-INF/jsp/cadastroFilme.jsp").forward(request,
27            response);
28        return;
29    }
30
31    int anoLancamento = anoLancamentoStr == null ? 0 : Integer.parseInt(
32        anoLancamentoStr);
33
34    try {
35        if (filmeDAO.existeComTituloEAno(titulo, anoLancamento)) {
36            request.setAttribute("erros", List.of(
37                "Já existe um filme cadastrado com esse título e ano de lançamento."
38            ));
39            request.getRequestDispatcher("/WEB-INF/jsp/cadastroFilme.jsp").forward(
40                request, response);
41            return;
42        }
43    } catch (SQLException e) {
44        getServletContext().log("Falha ao verificar duplicidade de Filme", e);
45        request.setAttribute("erros", List.of(
46            "Não foi possível salvar o filme agora. Tente novamente em instantes."));
47        ;
48        request.getRequestDispatcher("/WEB-INF/jsp/cadastroFilme.jsp").forward(request,
```



```

        response);
41     return;
42 }
43
44 Filme filme = new Filme();
45 filme.setTitulo(titulo);
46 filme.setDiretor(diretor);
47 filme.setAnoLancamento(anoLancamento);
48 filme.setGenero(genero);
49 filme.setSinopse(sinopse);
50 filme.setCapaUrl(capaUrl);
51
52 try {
53     filmeDAO.inserir(filme);
54 } catch (SQLException e) {
55     // Log server-side; usuário recebe só uma mensagem amigável (nunca a exceção
56     // crua).
57     getServletContext().log("Falha ao inserir Filme", e);
58     request.setAttribute("erros", List.of(
59         "Não foi possível salvar o filme agora. Tente novamente em instantes."));
60     ;
61     request.getRequestDispatcher("/WEB-INF/jsp/cadastroFilme.jsp").forward(request,
62         response);
63     return;
64 }
65
66 HttpSession session = request.getSession();
67 session.setAttribute("mensagemSucesso", "Filme \"" + titulo + "\" cadastrado com
68     sucesso!");
69 response.sendRedirect(request.getContextPath() + "/listarFilmes");
70 }

```

Código-Fonte 2: CadastrarFilmeServlet.doPost – validação, checagem de duplicidade e persistência

Destaco, no trecho acima, o padrão que repeti em toda a aplicação: (1) os dados de entrada passam por `FilmeValidator.tratarEntrada()` (normaliza *strings* vazias/apenas espaço para `null`) antes de qualquer outra lógica; (2) a validação (`FilmeValidator.validar()`) ocorre *antes* de qualquer tentativa de persistência, reexibindo o formulário com os valores já digitados em caso de erro – nunca uma perda de dados do usuário; (3) toda `SQLException` é capturada, registrada no log do servidor (`getServletContext().log()`) e nunca repassada ao usuário como texto de erro técnico, implementando diretamente a segunda situação-problema do enunciado (tratamento de exceções); e (4) a checagem de duplicidade evita registros redundantes tanto no cadastro manual quanto na importação via TMDb, sem custo adicional de complexidade (uma única consulta `SELECT` antes do `INSERT`).

4.3 Funcionalidade em destaque: Busca de Filme

A busca por título ou diretor é o cenário motivador direto da primeira situação-problema do enunciado (SQL Injection ao concatenar campos de busca). Assim implementei essa busca em `FilmeDAOImpl`:

```

1 // SQL_BUSCAR_POR_TITULO_OU_DIRETOR =
2 // "SELECT id, titulo, diretor, ano_lancamento, genero, sinopse, capa_url, nota_tmdb
   // +

```

```

3 // "FROM filme WHERE titulo LIKE ? OR diretor LIKE ? ORDER BY titulo";
4
5 @Override
6 public List<Filme> buscarPorTituloOuDiretor(String termo) throws SQLException {
7     List<Filme> filmes = new ArrayList<>();
8     String termoComCuringas = "%" + termo + "%";
9
10    try (Connection conexao = FabricaDeConexoes.getConexao();
11         PreparedStatement stmt = conexao.prepareStatement(
12             SQL_BUSCAR_POR_TITULO_OU_DIRETOR)) {
13
14        stmt.setString(1, termoComCuringas);
15        stmt.setString(2, termoComCuringas);
16
17        try (ResultSet rs = stmt.executeQuery()) {
18            while (rs.next()) {
19                filmes.add(mapearFilme(rs));
20            }
21        }
22
23    return filmes;
24 }

```

Código-Fonte 3: FilmeDAOImpl.buscarPorTituloOuDiretor – LIKE via PreparedStatement

O ponto central desta implementação, e a razão pela qual a escolhi como uma das duas funcionalidades detalhadas neste relatório, é a ordem das operações: o termo de busca do usuário é combinado com os curingas % (formando "%termo%") *antes* de ser passado como parâmetro via `stmt.setString()` – em nenhum momento o valor do termo é concatenado diretamente na *string* SQL em si (a *string* SQL, com seus dois ?, é uma constante fixa, definida uma única vez em `SQL_BUSCAR_POR_TITULO_OU_DIRETOR`). Essa distinção – concatenar o valor do parâmetro versus concatenar a consulta – é exatamente o que separa uma implementação segura de LIKE de uma vulnerável a SQL Injection. Toda a consulta roda em *try-with-resources*, garantindo o fechamento de conexão, *statement* e *result set* mesmo em caso de exceção, consistente com o padrão de tratamento de exceções que adotei em toda a camada de acesso a dados do projeto.

4.4 Extensões opcionais: integração TMDb e reestilização com Bootstrap

Além do escopo mandatório, implementei duas extensões opcionais mencionadas no enunciado da disciplina como possíveis, mas não obrigatórias. A primeira, integração com a API pública do TMDb (pacote `com.catalogo.tmdb`), permite: (a) descobrir filmes populares em um carrossel, com importação direta ao catálogo local; e (b) buscar um filme por título dentro do próprio formulário de cadastro, pré-preenchendo os campos – incluindo o pôster – a partir do resultado escolhido. A chave de API nunca é *hardcoded* (segue o mesmo padrão de `db.properties` usado para as credenciais de banco), toda chamada HTTP tem *timeout* curto, e qualquer falha da API externa é isolada em uma exceção própria (`TmdbException`), sem nunca afetar o cadastro/edição/listagem manuais do catálogo. A segunda extensão reestiliza toda a interface com o *framework* Bootstrap 5 (via CDN, com *Subresource Integrity* – ver Seção 6), incluindo um modo escuro alternável

persistido em `localStorage` e efeitos visuais sutis (*fade-in*, realce ao passar o mouse), sem introduzir nenhuma dependência de *build* de front-end adicional.

5 Resultados

Nesta seção, apresento capturas de tela reais da aplicação em execução localmente, demonstrando as funcionalidades mandatórias do CRUD/busca e as duas extensões opcionais descritas na Seção 4.

5.1 Listagem de filmes

A tela inicial (`/listarFilmes`) exibe todos os filmes cadastrados, com título, diretor, ano, gênero e pôster (quando disponível), além do campo de busca por título/diretor no topo.

 Catálogo de Filmes

CatálogoCadastrarDescobrir (TMDB)



Filmes Catalogados

Buscar por título ou diretor

Buscar

Cadastrar novo filme

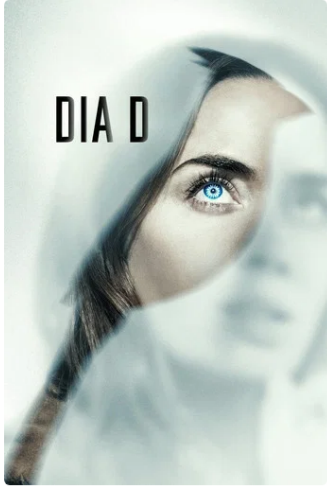
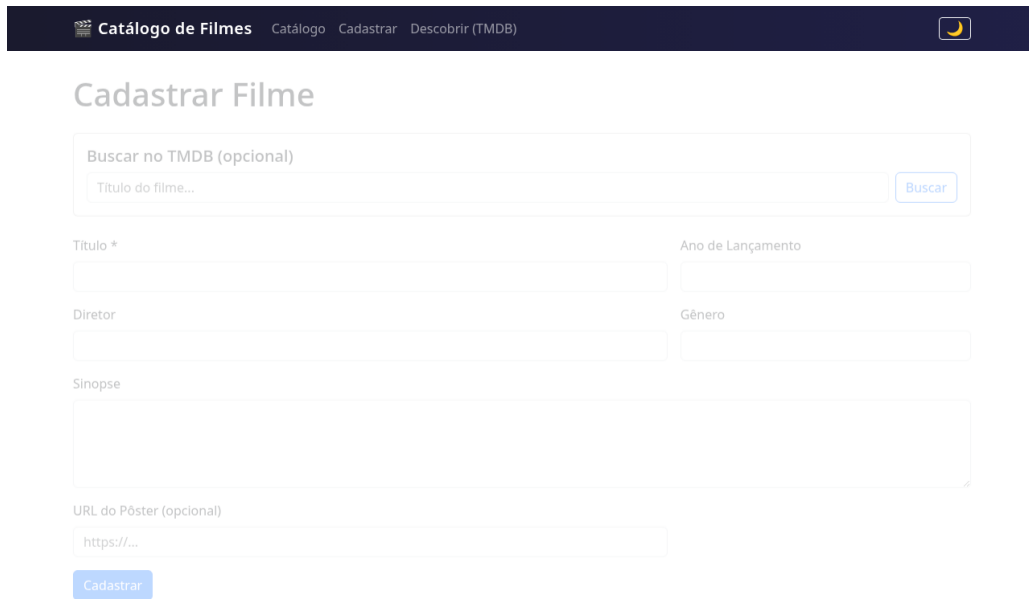
Pôster	Título	Diretor	Ano	Gênero	Nota TMDb	
	A Odisseia	Christopher Nolan	2026	Aventura	8.0/10	Ver detalhes
	Dia D	Steven Spielberg	2026	Ficção científica	7.5/10	Ver detalhes
						

Figura 1: Listagem do catálogo (/listarFilmes).

5.2 Cadastro de filme

O formulário de cadastro (`/cadastrarFilme`), com o campo obrigatório de título destacado e o campo opcional de busca por título no TMDb.



The screenshot shows a web application interface for adding a movie. At the top is a dark navigation bar with the text 'Catálogo de Filmes' and links for 'Catálogo', 'Cadastrar', and 'Descobrir (TMDb)', along with a moon icon. Below the navigation bar, the page title 'Cadastrar Filme' is displayed. The form itself is a light gray box containing several input fields: a search section with the label 'Buscar no TMDb (opcional)', a text input for 'Título do filme...', and a 'Buscar' button; a 'Título *' field (marked as required); an 'Ano de Lançamento' field; a 'Diretor' field; a 'Gênero' field; a large 'Sinopse' text area; and a 'URL do Pôster (opcional)' field with a placeholder 'https://...'. A 'Cadastrar' button is located at the bottom left of the form.

Figura 2: Formulário de cadastro (`/cadastrarFilme`), vazio.

5.3 Detalhe de filme

A página de detalhe (`/detalharFilme?id=N`) exibe todos os campos de um filme específico – no exemplo a seguir, o filme “A Odisseia” – com links para editar e excluir.

 Catálogo de Filmes

CatálogoCadastrarDescobrir (TMDB)





A ODISSEIA

A Odisseia

Nota TMDB: 8.0/10

Diretor: Christopher Nolan

Ano de Lançamento: 2026

Gênero: Aventura

Sinopse:

Acompanhe a saga de Odisseu, o lendário rei de Ítaca, em sua longa e perigosa jornada de retorno ao lar após a Guerra de Troia.

[Editar](#)[Excluir](#)

Figura 3: Página de detalhe de um filme (/detalharFilme?id=10).

5.4 Edição de filme

O formulário de edição (`/editarFilme?id=N`) vem pré-preenchido com os dados atuais do filme – no exemplo, “Tropa de Elite”, já com pôster importado do TMDb visível.

Catálogo de Filmes

CatálogoCadastrarDescobrir (TMDb)

Editar Filme

Título *

Tropa de Elite

Ano de Lançamento

2007

Diretor

José Padilha

Gênero

Drama

Sinopse

Rio de Janeiro, 1997. Nascimento, capitão da Tropa de Elite do Rio de Janeiro, é designado para chefiar uma das equipes que tem como missão "apaziguar" o Morro do Turano por um motivo que ele considera insensato. Mas ele tem que cumprir as ordens enquanto procura por um substituto. Sua mulher, Rosane, está no final da gravidez e todos os dias lhe pede para sair da linha de frente do batalhão. Pressionado, o capitão sente os efeitos do estresse. Neste clima, é chamado para mais uma emergência num morro. Em meio a um tiroteio em um baile funk,

URL do Pôster (opcional)

<https://image.tmdb.org/t/p/w342/jpG8AVzRQabzjHeJ7jWbuXe9T1n.jpg>



Salvar

Figura 4: Formulário de edição pré-preenchido (`/editarFilme?id=12`).

5.5 Extensão: Descobrir filmes (TMDb)

O carrossel de filmes populares (`/descobrirFilmes`), consumindo a API pública do TMDb em tempo real, com botão de importação direta ao catálogo.

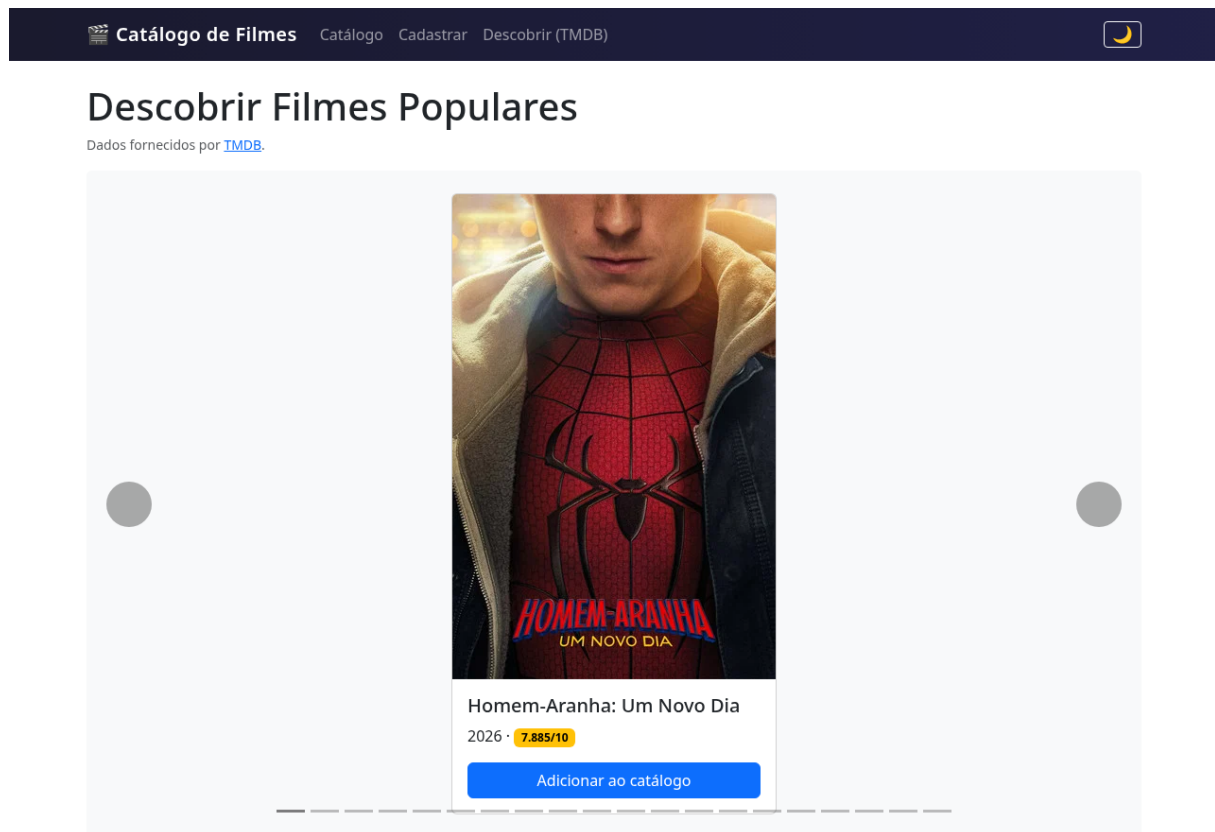


Figura 5: Carrossel de filmes populares do TMDB (/descobrirFilmes).

5.6 Extensão: busca no TMDb dentro do cadastro

Busca por título consumindo o TMDb diretamente de dentro do formulário de cadastro, exibindo até cinco resultados para o usuário escolher e pré-preencher o formulário.

The screenshot shows a web application interface for adding a movie. At the top, a dark navigation bar contains the text 'Catálogo de Filmes' and links for 'Catálogo', 'Cadastrar', and 'Descobrir (TMDb)', along with a moon icon. Below this, the page title 'Cadastrar Filme' is displayed. The main form is titled 'Cadastrar Filme' and includes a section for searching on TMDb. This section has a text input field containing 'Duna' and a 'Buscar' button. Below the search bar, five movie posters are shown as search results, each with a 'Usar este' button underneath. The results are: 'Duna (2021)', 'Duna: Parte Dois (2024)', 'Duna (1984)', 'Duna: Parte Três (2026)', and 'Duna (2018)'. Below the search results, the form continues with fields for 'Título *', 'Ano de Lançamento', 'Diretor', and 'Gênero'. There is a large text area for 'Sinopse' and a field for 'URL do Pôster (opcional)' with a placeholder 'https://...'. A 'Cadastrar' button is at the bottom of the form.

Figura 6: Busca por título no TMDb, dentro do cadastro (/cadastrarFilme?buscaTmdb=Duna).

6 Considerações sobre Segurança

Tratei a segurança como parte integral do desenvolvimento (princípio de *Secure by Design*, McGRAW, 2006): apliquei uma revisão de segurança dedicada, seguindo um checklist fixo, ao final de cada funcionalidade – **antes** de qualquer especificação ser considerada concluída – e, após a conclusão de todas as funcionalidades mandatórias, executei uma trilha independente de teste de penetração (*pentest*) *blackbox*, escopada pelo OWASP Top 10:2021.

6.1 Revisão de segurança por funcionalidade

Submeti todas as especificações do projeto (cadastro, listagem, detalhe, edição, exclusão, busca, integração TMDb e reestilização de interface) a uma revisão de segurança formal, cobrindo cinco categorias fixas: SQL Injection, tratamento de exceções, validação de dados, XSS (*Cross-Site Scripting*) e proteção de credenciais/chaves. Os pontos centrais, consistentes em todo o projeto, são:

- **SQL Injection (situação-problema 1):** todo acesso ao banco de dados, em `FilmeDAOImpl`, usa `PreparedStatement` com parâmetros ? – nunca concatenação de *string* SQL com entrada do usuário, mesmo dentro de cláusulas `LIKE` (ver Listagem 3);
- **Tratamento de exceções (situação-problema 2):** todo acesso JDBC roda em *try-with-resources*; toda `SQLException` é registrada no log do servidor e nunca exposta ao usuário como *stack trace* ou mensagem técnica – o usuário sempre recebe uma mensagem amigável e genérica;
- **Validação de dados:** centralizada em `FilmeValidator`, aplicada *antes* de qualquer persistência – título obrigatório, ano de lançamento numérico quando informado, tamanhos máximos de campo alinhados à DDL da tabela `filme`;
- **XSS:** toda saída de dado originado do usuário (ou do TMDb) nas páginas JSP usa a tag JSTL `<c:out>`, que escapa automaticamente marcação HTML/JavaScript – nunca Expression Language (EL) crua (`${...}`) para texto livre de usuário. Isso cobre tanto XSS persistente (dados vindos do banco, como na listagem) quanto XSS refletido (como o termo de busca reexibido no campo de busca);
- **Credenciais e chaves:** as credenciais de banco de dados (`db.properties`) e a chave de API do TMDb (`tmdb.properties`) nunca são *hardcoded* no código-fonte, ficam fora do controle de versão (`.gitignore`) e são lidas exclusivamente por classes utilitárias dedicadas (`FabricaDeConexoes`, `FabricaTmdb`).

Obtive aprovação integral em todas as revisões (5/5 itens, com alguns itens marcados N/A quando não aplicáveis, como validação de dados em funcionalidades somente-leitura). As poucas observações que registrei durante essas revisões classifiquei como não bloqueantes – lacunas de robustez defensiva, não vulnerabilidades de segurança propriamente ditas (por exemplo, ausência de validação de faixa de valores em um campo interno que nunca é editado diretamente pelo usuário) – e as documentei nas especificações correspondentes para eventual correção futura.

6.2 Achados do pentest *blackbox* (Nível 3 – trilha de segurança)

Após a conclusão de todas as funcionalidades mandatórias, executei uma trilha independente de teste de penetração *blackbox*, escopada pelo OWASP Top 10:2021, contra a aplicação em execução. Registrei dois achados, ambos já triados e **encerrados**:

- **Ausência de controle de acesso e de proteção CSRF** (severidade Média, OWASP A01:2021 – *Broken Access Control*). Como a aplicação não implementa autenticação – decisão de escopo consistente com o enunciado, que descreve um catálogo de uso pessoal –, qualquer visitante que descubra um `id` de filme pode, em

tese, editá-lo ou excluí-lo, e o formulário de exclusão não possui token anti-CSRF (a única mitigação é o uso exclusivo de POST, nunca GET, mais uma confirmação do lado do cliente). Eu já havia identificado esse indício e o havia formalmente aceito como risco durante as revisões de segurança de cada especificação individual; o pentest *blackbox* apenas confirmou, de fora, o que eu já sabia. Status final: **Risco Aceito** – pretendo reavaliar caso implemente uma extensão de autenticação no futuro;

- **Ausência de cabeçalhos HTTP de segurança** (severidade Baixa, OWASP A05:2021 – *Security Misconfiguration*). A configuração padrão do servidor não incluía cabeçalhos de defesa em profundidade como X-Frame-Options, Content-Security-Policy, X-Content-Type-Options e um atributo SameSite explícito no cookie de sessão. Eu **corrigi** efetivamente este achado: implementei um novo filtro de *servlet*, `CabecalhosSegurancaFilter`, que adiciona os quatro primeiros cabeçalhos a toda resposta HTTP (com uma *Content Security Policy* restrita aos domínios efetivamente usados pela aplicação – CDN do Bootstrap e imagens do TMDb), e configurei o cookie de sessão com SameSite=Lax via META-INF/context.xml do Tomcat. Revalidei a correção (mvn test, inspeção dos cabeçalhos HTTP reais e confirmação de que nenhuma funcionalidade existente foi quebrada). Status final: **Corrigido**.

6.3 Cobertura de testes

Escrevi, para a camada de acesso a dados (FilmeDAOImpl), uma suíte de 20 testes JUnit 5 (src/test/java/com/catalogo/dao/FilmeDAOTest.java), que executei de fato (mvn test) contra um banco MariaDB real, cobrindo todos os métodos de FilmeDAO – incluindo casos de sucesso, ausência de correspondência, e um teste específico de sondagem com aspas simples (buscarPorTituloOuDiretor_naoDeveLancarExcecao_comAspasSimples) para confirmar, empiricamente, a ausência de vulnerabilidade a SQL Injection na funcionalidade de busca.

7 Conclusões

7.1 Dificuldades encontradas

As maiores dificuldades técnicas que enfrentei não vieram do CRUD em si – a arquitetura em camadas (JSP/Servlet/DAO) é direta uma vez internalizada –, mas de detalhes de ambiente e do comportamento do próprio contêiner de Servlets, difíceis de prever sem testar a aplicação de ponta a ponta. Dois exemplos concretos: (1) minha primeira tentativa de subir a aplicação no Tomcat falhou com “No suitable driver found”, porque `FabricaDeConexoes` não registrava explicitamente o driver JDBC (`Class.forName("com.mysql.cj.jdbc.Driver")`) – o driver estava no WAR, mas isso não bastava; e (2) acentos ficavam corrompidos (“Ficção” virava “FicÃ§Ã£o”) porque o Tomcat decodifica parâmetros de formulário como ISO-8859-1 por padrão, resolvido com um filtro dedicado (`CodificacaoUtf8Filter`) – e descobri, na sequência, que um fragmento JSP incluído via `<%@ include %>` não herda automaticamente o `pageEncoding` da página principal, então precisei repetir a diretiva em cada fragmento. Outra dificuldade, já numa fase mais avançada, foi perceber que eu mesmo tinha introduzido um bug de duplicidade: nem o cadastro manual nem a importação via TMDb verificavam se o mesmo

filme já existia antes de inserir, o que só ficou evidente testando a aplicação de verdade (encontrei “A Odisseia” e “Homem-Aranha: Sem Volta Para Casa” duplicados no meu próprio catálogo) – corrigido com uma checagem de duplicidade por título e ano antes do `INSERT (FilmeDAO.existeComTituloEAno)`.

7.2 Aprendizados

Tecnicamente, o maior aprendizado foi ver como pequenas decisões de arquitetura evitam duplicação de código de verdade: centralizar a validação em `FilmeValidator` significou que o cadastro manual e a importação do TMDb puderam compartilhar a mesma lógica de validação e de checagem de duplicidade, em vez de eu reimplementá-la duas vezes. Aprendi também, na prática, por que `PreparedStatement` e *try-with-resources* não são só “boas práticas” abstratas: escrever a busca por título/diretor com `LIKE` me obrigou a pensar exatamente onde o dado do usuário entra na consulta (como parâmetro, nunca concatenado à *string SQL*), e testar isso empiricamente com um valor de sondagem (‘) me deu confiança real, não só teórica, de que a implementação era segura. Do ponto de vista de processo, seguir um fluxo de especificação antes de código – e uma revisão de segurança formal antes de considerar qualquer funcionalidade concluída – tornou o pentest *blackbox*, ao final, quase um anticlímax: dos dois achados registrados, um já era um risco que eu mesmo tinha identificado e aceito conscientemente durante o desenvolvimento, e o outro (cabecinhos HTTP de segurança ausentes) era uma lacuna de defesa em profundidade, não uma vulnerabilidade explorável. Isso me convenceu de que revisar segurança continuamente, funcionalidade por funcionalidade, é mais eficaz do que deixar tudo para uma auditoria única no final.

7.3 Trabalhos futuros

O catálogo, hoje, é uma ferramenta de uso pessoal: um único “usuário” implícito, sem contas, sem distinção entre quem cadastrou o quê. O trabalho futuro que mais me interessa é justamente romper essa limitação e evoluir o projeto de um catálogo pessoal para uma pequena **comunidade de cinema**, inspirada em serviços como o Letterboxd: cada pessoa com sua própria conta e perfil, cadastrando os filmes que assistiu, escrevendo resenhas próprias – com nota pessoal, numa escala definida por mim (por exemplo, 0 a 5 estrelas), distinta da `notaTmdb` pública já existente – e interagindo com as resenhas de outras pessoas: curtir, comentar, seguir outros perfis e ver um feed de atividade de quem você segue.

Do ponto de vista técnico, isso significa uma mudança de escopo relevante, não só uma funcionalidade a mais. Eu precisaria introduzir autenticação de usuários – o que reabriria, com peso muito maior, a discussão de controle de acesso da Seção 6, hoje aceita como risco justamente porque não há contas de usuário –, uma nova entidade `Usuario` e uma entidade `Resenha` (ligando `Usuario` e `Filme`, com nota pessoal, texto e data), além de uma relação `Usuario-Usuario` para modelar “seguir”. Cada resenha passaria a ter um dono, e as operações de editar/excluir precisariam verificar explicitamente que o usuário autenticado é o dono do recurso – não mais qualquer visitante podendo alterar qualquer registro, como acontece hoje. Escalar dessa forma também levantaria questões que o projeto atual não precisa responder: paginação da listagem e do feed (o catálogo hoje é pequeno o suficiente para carregar tudo de uma vez), índices adicionais no banco de dados para as novas chaves estrangeiras, e, possivelmente, cache das chamadas ao TMDb para

não repetir a mesma consulta externa toda vez que várias pessoas revisarem o mesmo filme. É um passo natural depois deste projeto acadêmico – e, na minha opinião, o que tornaria o catálogo genuinamente útil além do uso pessoal que já faço dele hoje.

8 Referências

8.1 Bibliografia da disciplina

- BENDER, W. N. *Aprendizagem baseada em projetos: educação diferenciada para o século XXI*. 1. ed. Porto Alegre: Penso, 2014.
- BLUMENFELD, P. C. et al. Motivating project-based learning: Sustaining the doing, supporting the learning. *Educational Psychologist*, [s.l.], v. 26, n. 3-4, p. 369-398, 1991. Disponível em: <https://doi.org/10.1080/00461520.1991.9653139>. Acesso em: 05/04/2025.
- BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. *UML: guia do usuário*. 2. ed. Rio de Janeiro: Elsevier, 2007.
- DEITEL, P.; DEITEL, H. *Java®: como programar*. 10. ed. São Paulo: Pearson Universidades, 2017.
- FOWLER, M. *UML distilled: a brief guide to the standard object modeling language*. 3rd ed. Boston: Addison-Wesley Professional, 2004.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. *ISO/IEC 27001:2013: Information technology – Security techniques – Information security management systems – Requirements*. 1st ed. Geneva: ISO, 2013.
- KNUTH, D. E. Computer programming as an art. *Communications of the ACM*, v. 17, n. 12, p. 667-673, Dec. 1974. Disponível em: <https://doi.org/10.1145/361604.36161>. Acesso em: 05/04/2025.
- LARMER, J.; MERGENDOLLER, J.; BOSS, S. *Setting the standard for project-based learning: a proven approach to rigorous classroom instruction*. 1st ed. Alexandria: ASCD, 2015.
- MCGRAW, G. *Software security: building security in*. 1st ed. Upper Saddle River: Addison-Wesley, 2006.
- MYERS, G. J.; SANDLER, C.; BADGETT, T. *The art of software testing*. 3rd ed. Hoboken: John Wiley & Sons, 2012.
- OLIVEIRA, K. Aprendizagem Baseada em Projetos como estratégia para o desenvolvimento de atividades não presenciais no ensino médio integrado em informática no IFB Campus Brasília. *Revista Brasileira da Educação Profissional e Tecnológica*, v. 2, n. 22, e11815, 2022 (CC BY 4.0) | ISSN 2447-1801 | DOI: <https://doi.org/10.15628/rbept.2022.11815>. Acesso em: 05/04/2025.
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.

- THOMAS, J. W. *A review of research on project-based learning*. 1st ed. San Rafael: Autodesk Foundation, 2000. Disponível em: http://www.bobpearlman.org/BestPractices/PBL_Research.pdf. Acesso em: 25/05/2025.
- TRILLING, B.; FADEL, C. *21st century skills: learning for life in our times*. 1st ed. San Francisco: Jossey-Bass, 2009.

8.2 Documentação técnica consultada

- ORACLE. *Java Platform, Standard Edition Documentation*. Disponível em: <https://docs.oracle.com/en/java/javase/>.
- ORACLE. *The Java Tutorials – JDBC Database Access*. Disponível em: <https://docs.oracle.com/javase/tutorial/jdbc/>.
- OWASP FOUNDATION. *OWASP Top 10:2021*. Disponível em: <https://owasp.org/Top10/>.
- THE MOVIE DATABASE (TMDB). *API Documentation*. Disponível em: <https://developer.themoviedb.org/docs>.
- THE BOOTSTRAP AUTHORS. *Bootstrap 5 Documentation*. Disponível em: <https://getbootstrap.com/docs/5.3/>.
- ORACLE. *How to Write Doc Comments for the Javadoc Tool*. Disponível em: <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>.

9 Documentação do Código-Fonte

Comentei todo o código-fonte Java do projeto de duas formas complementares. Primeiro, comentários de linha (//) explicam decisões de design pontuais e blocos de lógica não óbvia diretamente no ponto onde ocorrem – por exemplo, a razão pela qual o filtro de codificação UTF-8 precisa ser o primeiro da cadeia de filtros, ou por que o campo `notaTmdb` é do tipo `Double` (objeto) em vez de `double` (primitivo). Segundo, e mais formalmente, toda classe pública e todo método público de `com.catalogo.dao`, `com.catalogo.servlet` e `com.catalogo.service` possuem comentários **Javadoc** (`/** ... */`), seguindo a convenção padrão da linguagem Java, incluindo descrição do propósito, parâmetros (`@param`), retorno (`@return`) e exceções lançadas (`@throws`) quando aplicável.

A interface `FilmeDAO` e a classe `FabricaDeConexoes` já continham Javadoc completo desde a implementação original de cada funcionalidade. Nesta rodada final de documentação, auditei e complementei o Javadoc nos métodos `doGet`/`doPost` de todos os sete Servlets do projeto e no método `doFilter` dos dois filtros de *servlet* (`CodificacaoUtf8Filter` e `CabecalhosSegurancaFilter`), que já possuíam Javadoc a nível de classe, mas ainda não a nível de método – nove classes ao todo passaram a ter Javadoc de método, descrevendo especificamente a rota/operação HTTP tratada por cada uma, seus parâmetros de requisição e o comportamento em caso de erro. No Javadoc de `com.catalogo.model.Filme`, segui a recomendação da disciplina de manter um comentário de classe curto (sem Javadoc redundante em cada *getter/setter* trivial).

10 Manual do Usuário Simplificado

Escrevi um manual do usuário simplificado, voltado a quem nunca utilizou o sistema (linguagem não técnica), disponível separadamente em `docs/manual-usuario.md`. Ele cobre, em uma a duas páginas, como acessar a aplicação, cadastrar um novo filme (incluindo o uso opcional da busca por título no TMDb), listar e buscar filmes já cadastrados, visualizar os detalhes de um filme, editá-lo e excluí-lo, além de uma breve seção de dúvidas frequentes – entre elas, por que o sistema não exige login (decisão de escopo já discutida na Seção 6 deste relatório).